

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 10%

QUESTIONS: 2

Duration: 45 Minutes
Total Marks:20

Annexure A

Descriptive Written Test

Code of Conduct:

*I C. PRIYANKA / 192511074 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: C. PRIYANKA

Descriptive Written Test

1. A smart home automation system is being designed to manage lighting, temperature, and security based on user behavior and environmental conditions. The system must respond to real-time inputs such as motion detection, temperature changes, and time of day while also learning user preferences over time. In this context, explain the different types of intelligent agents (simple reflex, model-based, goal-based, and utility-based agents) and analyze how each type would function within this system. Justify which type of agent or hybrid approach would be most effective for ensuring comfort, energy efficiency, and security.

ANSWERS:

Aim

To study the different types of Intelligent Agents and analyze their role in a smart home automation system for improving comfort, energy efficiency, and security.

Objective

- To understand the different types of Intelligent Agents.
- To analyze how each agent functions in a smart home environment.
- To compare different agent models.
- To identify the most suitable intelligent agent for smart home automation.

Problem Statement

A smart home automation system is designed to control lighting, temperature, and security based on user behavior and environmental conditions. The system receives real-time inputs such as motion detection, temperature changes, and time of day, while learning user preferences over time.

The task is to explain the different types of intelligent agents (Simple Reflex, Model-Based, Goal-Based, and Utility-Based) and determine the most suitable approach for achieving comfort, energy efficiency, and security.

Solution

1. Simple Reflex Agent

- Works based on the current input only.
- Uses condition-action rules.

Example:

- If motion is detected → Turn ON the light.
- If no motion → Turn OFF the light.

Advantages

- Simple and fast.
- Suitable for basic automation.

Limitation

- Does not remember previous states.
- Cannot learn user preferences.

2. Model-Based Agent

- Maintains an internal model of the environment.
- Uses previous information for decision-making.

Example:

- Remembers whether a room is occupied.
- Adjusts lighting based on occupancy history.

Advantages

- Better decision making.
- Handles partially observable environments.

3. Goal-Based Agent

- Makes decisions to achieve a specific goal.

Example:

- Goal: Maintain room temperature at 24°C.
- Goal: Lock all doors before bedtime.

Advantages

- Chooses actions that help achieve the desired goal.
- More flexible than reflex agents.

4. Utility-Based Agent

- Selects the action that provides the highest overall benefit.
- Considers multiple factors before making a decision.

Example:

- Balance user comfort and electricity consumption.
- Reduce AC usage when electricity prices are high.

Advantages

- Optimizes comfort, security, and energy efficiency.
- Produces the best overall outcome.

Comparison of Intelligent Agents

<u>Agent Type</u>	<u>Function in Smart Home</u>
<u>Simple Reflex</u>	<u>Turns lights ON/OFF based on motion.</u>
<u>Model-Based</u>	<u>Remembers room occupancy and user behavior.</u>
<u>Goal-Based</u>	<u>Achieves goals like maintaining temperature and security.</u>
<u>Utility-Based</u>	<u>Chooses the best action by balancing comfort, energy, and security.</u>

Most Suitable Approach

A Hybrid Approach combining Model-Based and Utility-Based Agents is the most effective.

Reason:

- Learns user preferences over time.
- Responds to real-time sensor inputs.
- Minimizes energy consumption.
- Maximizes user comfort.
- Improves home security.
- Makes intelligent decisions based on multiple factors.

Python Code:

```
# Smart Home Automation System
```

```
motion = input("Motion detected (yes/no): ")
temperature = int(input("Enter room temperature: "))
time = input("Time (day/night): ")
```

```
# Lighting
```

```
if motion == "yes":
    print("Light ON")
else:
    print("Light OFF")
```

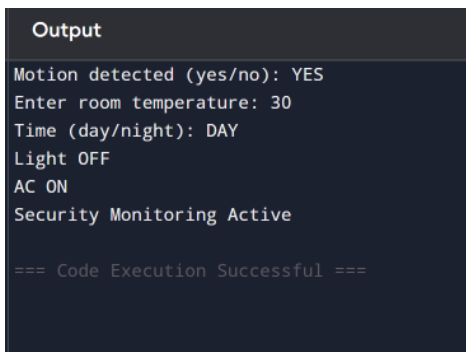
```
# Temperature Control
```

```
if temperature > 28:
    print("AC ON")
elif temperature < 20:
    print("Heater ON")
```

```
else:
    print("Temperature is Comfortable")

# Security
if time == "night" and motion == "no":
    print("Doors Locked")
else:
    print("Security Monitoring Active")
```

Sample Output:



```
Output
Motion detected (yes/no): YES
Enter room temperature: 30
Time (day/night): DAY
Light OFF
AC ON
Security Monitoring Active

=== Code Execution Successful ===
```

Discussion

A smart home requires intelligent decision-making because user preferences and environmental conditions continuously change. Simple Reflex Agents are suitable for basic tasks but cannot adapt. Model-Based Agents maintain information about previous states, while Goal-Based Agents focus on achieving predefined objectives. Utility-Based Agents evaluate multiple possible actions and select the one that provides the highest benefit. Therefore, combining Model-Based and Utility-Based Agents provides a more adaptive, efficient, and secure automation system.

Conclusion

The Smart Home Automation System can effectively manage lighting, temperature, and security using Intelligent Agents. Among the different agent types, a **Hybrid Model combining Model-Based and Utility-Based Agents** is the most suitable because it learns user preferences, responds intelligently to real-time conditions, minimizes energy consumption, and enhances security. This approach provides the highest level of comfort, energy efficiency, and reliable home automation.

2. A robot is placed in an unknown maze and must find a path to the exit without prior knowledge of the layout. Explain how uninformed search strategies like Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can help solve this problem.

Discuss the advantages and disadvantages of these algorithms in terms of time and space complexity. Relate the problem to different types of intelligent agents and explain how agent design affects decision-making. Suggest which combination of agent type and search strategy would be most effective and justify your choice.

ANSWER:

Aim

To study and compare **Uniform Cost Search (UCS)** and **Iterative Deepening Search (IDS)** for solving a robot maze problem and to analyze the role of intelligent agents in decision-making.

Objective

- To understand uninformed search strategies.
- To compare Uniform Cost Search (UCS) and Iterative Deepening Search (IDS).
- To analyze time and space complexity.
- To relate search algorithms with intelligent agent types.
- To identify the most suitable agent and search strategy for maze navigation.

Problem Statement

A robot is placed in an **unknown maze** and must find a path to the exit without prior knowledge of the maze layout.

The robot must:

- Explore the maze.
- Find the exit.
- Avoid unnecessary exploration.
- Make intelligent decisions during navigation.

The task is to explain how **Uniform Cost Search (UCS)** and **Iterative Deepening Search (IDS)** solve the problem, compare their performance, and determine the best combination of intelligent agent and search strategy.

Solution

1. Uniform Cost Search (UCS)

Uniform Cost Search expands the node with the **lowest path cost** first.

Working

- Starts from the initial position.
- Expands the least-cost node.
- Continues until the goal is found.

Advantages

- Finds the optimal path.
- Works well when movement costs are different.

Disadvantages

- Uses more memory.
- Slower for large mazes.

2. Iterative Deepening Search (IDS)

Iterative Deepening Search repeatedly performs Depth-First Search with increasing depth limits.

Working

- Searches depth 0.
- Then depth 1.
- Then depth 2.
- Continues until the exit is found.

Advantages

- Requires less memory.
- Suitable for unknown-depth problems.

Disadvantages

- Repeats node expansion.
- May take more time.

Comparison of UCS and IDS

Feature	UCS	IDS
Strategy	Lowest Cost First	Depth Limited Search
Complete	Yes	Yes
Optimal	Yes	Yes (Equal Costs)
Time Complexity	$O(b^d)$	$O(b^d)$
Space Complexity	High	Low
Memory Usage	More	Less

Intelligent Agents

Simple Reflex Agent

- Makes decisions based only on the current state.
- Cannot remember previous paths.

Model-Based Agent

- Stores explored paths and obstacles.

- Makes better decisions in unknown environments.

Goal-Based Agent

- Selects actions to reach the maze exit.

Utility-Based Agent

- Chooses the safest and shortest path based on cost and efficiency.

Best Combination

The most effective combination is:

Model-Based Goal-Based Agent + Uniform Cost Search (UCS)

Justification

- The Model-Based Agent remembers explored paths.
- The Goal-Based Agent focuses on reaching the exit.
- UCS guarantees the least-cost path.
- This combination improves navigation accuracy and avoids repeated exploration.

Python Code:

```
from heapq import heappush, heappop
```

```
graph = {
    'S': [('A', 1), ('B', 2)],
    'A': [('C', 2)],
    'B': [('D', 3)],
    'C': [('G', 2)],
    'D': [('G', 1)],
    'G': []
}
```

```
pq = [(0, 'S')]
visited = set()
```

```
while pq:
    cost, node = heappop(pq)
```

```
    if node in visited:
        continue
```

```
    visited.add(node)
```

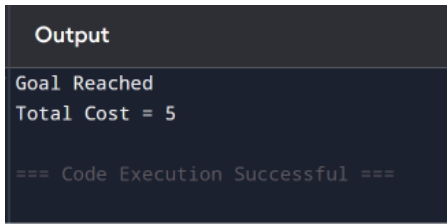
```
    if node == 'G':
```



```
print("Goal Reached")
print("Total Cost =", cost)
break
```

```
for next_node, next_cost in graph[node]:
    heappush(pq, (cost + next_cost, next_node))
```

Sample Output:



```
Output
Goal Reached
Total Cost = 5

=== Code Execution Successful ===
```

Discussion

In an unknown maze, the robot must explore while making intelligent decisions. UCS guarantees the shortest path when movement costs vary, whereas IDS is memory-efficient and suitable for limited-memory systems. Intelligent agent design also plays an important role. A Model-Based Agent can remember visited locations, and a Goal-Based Agent continuously plans actions toward the exit. Combining these with UCS results in reliable and efficient maze navigation

Conclusion

Uniform Cost Search and Iterative Deepening Search are effective uninformed search strategies for maze navigation. UCS provides the optimal path but requires more memory, whereas IDS uses less memory but may repeat searches. A Model-Based Goal-Based Intelligent Agent with Uniform Cost Search is the most suitable solution because it remembers explored paths, focuses on the goal, and finds the least-cost route efficiently. This approach ensures better decision-making and successful navigation in an unknown maze.